(a) Baseline: Shell Replacement (sed)

Blind regex string substitution without AST awareness

```
140 class QuerySet:
141     def filter(self, *args, **kwargs):
142         if key in clone.query: ...

    $ sed -i 's/clone.query/query/' queryset.py

    File "queryset.py", line 142
        if key in query:
        ^  SyntaxError!
    IndentationError: unexpected indent
    (expected 8 spaces, got 4)
```

Failure Mode: Indentation drift corrupts surrounding function scope. Command line lacks syntactic validation.

(b) Baseline: Unified Diff (w/o Linting)

Unconstrained diff hunks without AST scope anchoring

```
$ git apply candidate.patch
@@ -185,6 +185,8 @@ def filter(self, *args):
-     if key in clone.query:
+     parts = key.split('__')
+     if parts[0] in clone.query:

error: patch failed: queryset.py:188
error: queryset.py: patch does not apply
Hunk #2 FAILED at 188 (context mismatch).
1 out of 2 hunks FAILED (saved to .rej)
```

Failure Mode: Hallucinated line offsets and context mismatch across versions trigger strict hunk rejection.

(c) Proposed: Perron AST-Grounded Editor

AST-bounded search, indentation rebasing & pre-commit check

```
$ perron-edit --anchor="def filter" --range=[140,158]
[AST Match] Found QuerySet.filter at lines 140-158
[Rebase] Auto-adjusted base indentation: 8 spaces
[Pre-Commit] ast.parse() validated -> Syntax OK
[Commit] Safe write to disk. 0.0% Syntax Error.

$ pytest -k test_filter_rel
tests/test_filter.py::test_filter_rel PASSED [100%]
1 passed in 0.32s on CPU -> Git patch verified
Verified patch artifact ready for submission
```

Verification: bounded AST search and pre-commit parsing reject edits that do not parse